Ivar Boström 2202283

Problem Statement

The goal is to, given two random integer arrays A and B of size N, where N is very large, find the cheapest path through a matrix, where each position M[a][b] has a cost of A[a]+B[b], and the path begins at position M[0][0] ends at M[N-1][N-1] and each step can increase either a or b by one. This is a very slight variation on problem 917 on Project Euler. The differences are twofold. The original problem only cared about N=10000000, I am interested in larger numbers. Secondly the original problem begins with the creation of the arrays. While I also had to write code to do this, I begin timing only after their creation. Operating on the arrays is a more interesting and useful task than the array creation.

Why Optimize?

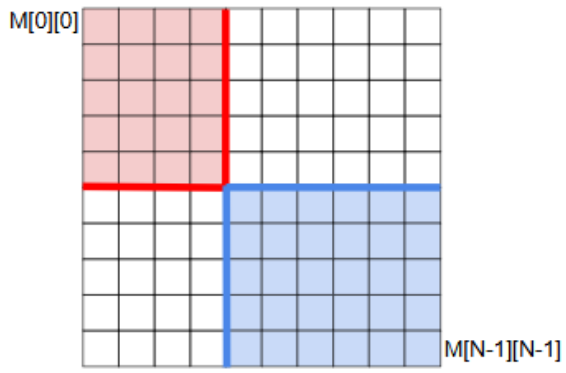
For large N, the matrix becomes impossible to materialize. Even if this is avoided, the search space is too large for a naive search algorithm. Memory can be kept linear, but computational complexity tends to be quadratic.

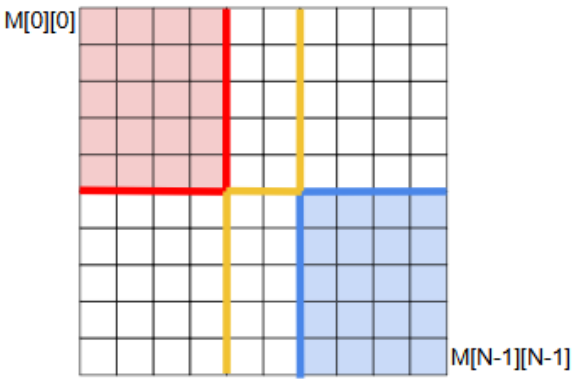
Optimization before Parallelization

First, rather than counting the cost of positions, we know that each column and row will be traversed at least once. Thus we can preemptively add the cost of each row and column. Then, while traversing, we instead add the cost for the column or row in which we are staying.

Another thing we can do before traversal, is to for each value in each array, subtract the smallest value of that array, and add the subtracted total to the base cost. After this normalization, the cheapest column and cheapest row become free to traverse. This is also true for tied cheapest.

In practice this means that the problem can be split into two smaller subproblems. At traversal, rather than traversing from start to finish, it is equivalent to traverse from start and finish onto either the cheapest row or the cheapest column. Whichever points they end on, the traversal cost between them is 0.

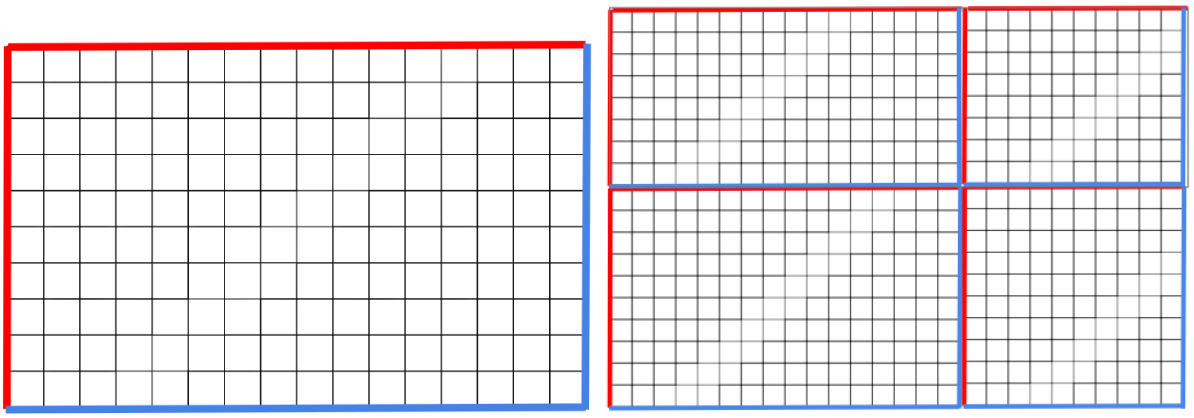




The traversal cost between any point on the red lines and any point on the blue lines is 0. Thus, our subproblems are in the blue and red areas respectively.

Lastly, and most crucially, we can reduce the searchspace of each subproblem. From the starting point, we may look only on the part of the grid up until the first column and row that are cheaper than the starting ones. We can then note that regardless of from which point we want to exit this part of the grid, the fastest way to get there begins with going straight until the boundary is reached. Once the boundary is reached however, this marks the starting point of a new such tile.

In other words, we are only interested in lines that are the cheapest column or row so far, for the given subproblem. But we are not only interested in their cost, but also the distance between them. This is the most significant realization, as for each subproblem, we statistically expect ln(N) relevant lines in each direction. From there any pathfinding algorithm will do. The complexity is dominated by the O(N) task of identifying the relevant lines in the first place, as other O(N) tasks, like summing.



Starting in the red corner, the cheapest way to any point on the blue lines goes through one of the red and blue corners. Which are themselves the red corner of the next tile.

Quick and Easy Pseudocode Implementation

```
cost=sum(A)+sum(B)+min(A)*(N-1)+min(B)*(N-1);
ca={A[0]}; cb={B[0]}; car={A[N-1]}; cbr={B[N-1]};
da={}; db={}; dar={}; dbr={};

p=0; m=A[0];
for (i in 1:N){ if (A[i]<m){
        m=A[i]; ca.append(m); da.append(i-p); p=i;
}}
p=0; m=B[0];
for (i in 1:N){ if (B[i]<m){
        m=B[i]; ca.append(m); da.append(i-p); p=i;
}}
p=0; m=A[0];
for (i in 1:N){ if (A[N-1-i]<m){
        m=A[N-1-i]; ca.append(m); da.append(i-p); p=i;
}}
p=0; m=B[0];
for (i in 1:N){ if (B[N-1-i]<m){
        m=B[N-1-i]; ca.append(m); da.append(i-p); p=i;
}}

AN=ca.size(); BN=cb.size();
m=infinity; d=m*[AN][BN]; d[0][0]=0;
for(a in 0:N){for(b in 0:N){
        if(a==AN-1 or b==BN-1){
                m = min(m,d[a][b]); continue;
        }
        d[a+1][b]=min(d[a+1][b], d[a][b]+da[a]*cb[b]);
        d[a][b+1]=min(d[a][b+1], d[a][b]+db[b]*ca[a]);
}}
cost+=m;

AN=car.size(); BN=cbr.size();
m=infinity; d=m*[AN][BN]; d[0][0]=0;
for(a in 0:N){for(b in 0:N){
        if(a==AN-1 or b==BN-1){
                m = min(m,d[a][b]); continue;
        }
        d[a+1][b]=min(d[a+1][b], d[a][b]+dar[a]*cbr[b]);
        d[a][b+1]=min(d[a][b+1], d[a][b]+dbr[b]*car[a]);
}}
cost+=m;
```

Adding Parallelization

Parallelizating over four threads is somewhat trivial. The heaviest step makes independent forwards and backwards passes over two arrays. Each of these can be their own thread. As a sidenote, calculating the sum, and finding the minima can be woven into these same passes.

For further parallelization, the arrays can be managed in chunks. If so however, a new section has to be included afterwards, which is itself limited to four threads. These are required to consolidate the chunks, as many of the values that locally qualified as the smallest so far, won't do so globally. Threading here saves minimal time, as the complexity of the step is ln(N).

The final pathfinding can also be parallelized over the two subproblems. The savings here are once again small, with the complexity being $\ln^2(N)$.

File Information and Instructions

The folder contains: MatrixTraversal, MatrixTraversalTiming, MatrixTraversalAccuracy, and MatrixTraversalComparison each as a C++ Source File, an Object File, and an Application. Compilation was performed through MSVC, with commands in the format:
`cl /O2 /openmp /std:c++17 /EHsc <name>.cpp /Fe:<name>.exe`
Running it would simply be: `<name>.exe` Different solutions may be applied in different environments.

MatrixTraversal simply outputs the cost for N=100000000. Increasing this value is probably possible up to a certain extent. 500000000 does not compile. 250000000 does compile, but my computer blocks me from running it. It is also possible that some values runnable on some machines would overflow an unsigned 64 bit integer. In these cases the code detects it.

MatrixTraversalAccuracy compares the values calculated to the correct values stated in the Project Euler problem. These however, are only for N=1, 2 and 10.

MatrixTraversalTiming runs computes the cost for N=100000000 100 times. And then 100 more without threading, outputting the average time for each, the total speedup with threads, and the speedup per thread. With 8 threads, my average time was 165ms, with one it was 645ms. This is roughly a 3.9x speedup, or a 0.5 speedup per thread. Computationally, threading should be nearly perfect, so the function is likely memory bound.

MatrixTraversalComparison compares runtime to a standard dynamic programming approach, for N=100,1000,10000. For me, the corresponding speedups were about 0.17, 146, and 8502. It is clear that the dynamic programming approach is superior for small input sizes, but falls behind quickly as N grows. We can extrapolate that the dynamic programming approach would take two years for N=100000000.

Example outputs:

MatrixTraversal:

```
998416621261419 37
```

MatrixTraversalAccuracy:

```
For N=1:  Expected:966774091 Got:966774091
For N=2:  Expected:2388327490 Got:2388327490
For N=10: Expected:13389278727 Got:13389278727
```

MatrixTraversalTiming:

```
Running 100 times, for 100000000*100000000 matrix. 8 Threads
Average time: 164.735672 ms
Running 100 times, for 100000000*100000000 matrix. 1 Thread
Average time: 645.312484 ms
Threads: 8 Speedup: 3.917260 Speedup/Thread: 0.489658
```

MatrixTraversalComparison:

```
N=100:   Solution: 0.681400ms Dynamic Programming: 0.116900ms Speedup: 0.171559
N=1000:  Solution: 0.050900ms Dynamic Programming: 7.419700ms Speedup: 145.770195
N=10000: Solution: 0.074600ms Dynamic Programming: 634.242200ms Speedup: 8501.907766
```

Github Link

https://github.com/Ivar-Bostrom/PPFP